Freight Rate Prediction: Validation Approach and Results

Abdul | Machine Learning Engineer assessment

1. The task and the data

The goal is to predict the posted rate for each of the 12,000 loads in validation.csv (November to December 2025) using 48,000 labeled loads from January to October 2025. Each load has a lane (pickup and delivery city with coordinates), distance, equipment type, weight, date, and two market signals (market_index and quote_signal). Distance explains most of the rate (correlation 0.91), so the work is mostly about the smaller effects on top of it.

2. Data quality issues and how I handled them

Issue	What I found	Handling
Negative weights	292 train and 145 validation rows; the absolute values match the normal weight range	Treated as a sign error, used the absolute value
Missing weight / market_index	About 0.6% and 0.8% of rows, spread evenly over time	Weight: training median. market_index: that day's median (it barely moves within a day). Missing flags added
Distance vs coordinates	Short lanes can be longer than the straight-line distance; there is a 70-mile minimum	Left alone, distance is reliable
Noisy rates	677 train rows (1.4%) are 2-5x too high or 3-5x too low, evenly spread over equipment, month and distance, and no feature explains them	Dropped from model fitting only, never from validation scoring

3. How I split and validated the data

validation.csv covers the two months after the training data ends, so the real job is forecasting forward in time. A random split would let the model see loads from the same days it is tested on, and the market signals are shared across a day, which would make the scores look better than they really are. So I used rolling, time-ordered folds: each fold trains only on earlier months and validates on the next two months.

- Folds: train Jan-Apr, validate May-Jun; train Jan-Jun, validate Jul-Aug; train Jan-Aug, validate Sep-Oct.
- Cleaning values (weight median) come from training data only. Day-level features use only feature columns, never rates.
- Noisy rows are removed from the training part of each fold, but the validation part keeps every row. Because they are impossible to predict, I report error on all rows and on the typical rows (error under 30%) separately.
- The final model is refit on all ten months before predicting validation.csv.

4. Features and model

Beyond the raw columns, the features that mattered were: log distance and short-haul flags; the flatbed premium that builds up through the last month of each quarter (March, June, September, so it also applies to December); the gap of quote_signal from its middle value, since both very low and very high values sit above average; and a 7-day average of the market index plus the daily mean quote signal. I tried a loads-per-day feature and dropped it, because validation has about 25% more loads per day than training and the model would not transfer.

The final model predicts log(rate) with an even blend of a linear regression and gradient boosting (scikit-learn HistGradientBoostingRegressor, 15 leaves, min 100 rows per leaf, squared error loss). The linear model captures

the smooth multiplicative structure well, boosting picks up the curves and interactions, and the blend was the most consistent across folds. I tried 12 boosting settings and 4 blend weights; the top settings were within about 1-2% of each other, so the result is not sensitive to tuning.

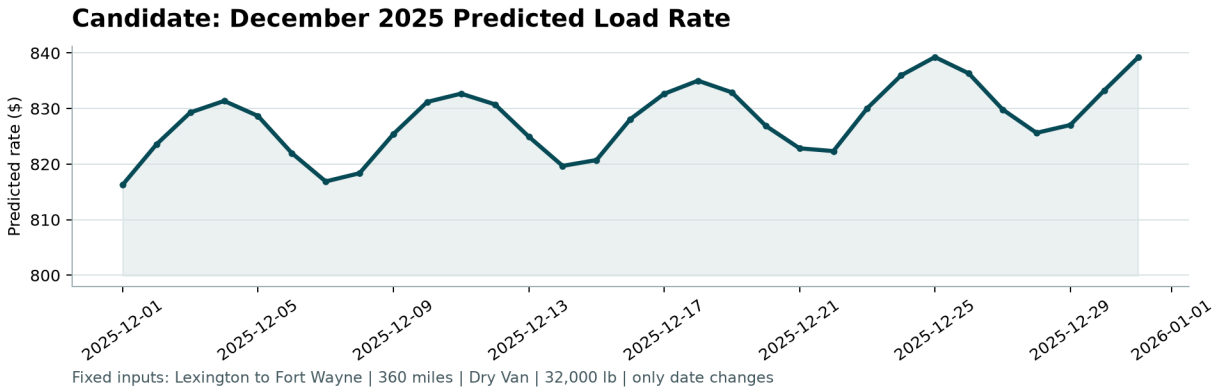
5. Validation results

Fold (train -> validate)	Train rows	Valid rows	Baseline MedAPE	Model MedAPE	MAE (typical)	MAPE (typical)	MAE (all)
Jan-Apr -> May-Jun	18,835	9,696	7.1%	1.49%	$45	1.77%	$99
Jan-Jun -> Jul-Aug	28,406	9,671	6.7%	1.76%	$49	2.14%	$101
Jan-Aug -> Sep-Oct	37,944	9,523	6.8%	1.80%	$48	2.01%	$104

The baseline is a median rate per mile by equipment type times distance. MAE (all) is higher than MAE (typical) because of the unpredictable noisy rows, which also keep RMSE around $630 for every model, so I do not use RMSE to compare models.

6. Fixed December 2025 prediction chart

The chart file only fixes the lane (Lexington to Fort Wayne), distance, equipment, and weight, so the model also needs the market signals for each day. I took the daily median market_index and quote_signal from the December loads in validation.csv and used the same day-level context features. The curve is produced by score.py.



Limits: the training data has no December, so the seasonal level is extrapolated through the market signals and the quarter feature. Holidays are not modeled, so the Christmas week should be read with care.